

**A report submitted in partial fulfilment of the regulations governing the award of
the Degree of Games Computing**

Project Report

A Comparison Between Behaviour Trees and Utility AI Decision-Making Systems in Games.

2025/2026

Student Name: Callum Christer

Supervisor Name: Dr Martin Wonder

Abstract

The aim of this investigation is to compare the impact of behaviour trees and utility AI on non-player character (NPC) decision-making and adaptability in Unreal Engine 5 (Epic Games, 2025). Despite their widespread use in game development and extensive individual literature, there is limited comparative research evaluating their effectiveness on NPC decision-making. To investigate this fifty, five-minute simulations for each AI framework were conducted within a survival-based environment, developed with separate identical NPCs controlled by either behaviour trees or utility AI.

To measure the impact of behaviour trees and utility AI, metrics were produced and recorded during comparative testing including task efficiency, task engagement rate and overall survivability. A comprehensive metric, the NPC Appropriate Behaviour and Adaptability Score (NABAS) was also created to provide a unified measurement of performance.

The results show a clear difference between the frameworks. Behaviour trees achieved substantially higher task efficiency scoring 96.49%, emphasising their ability to encourage predictable behaviour, but performed worse in survivability only surviving 32% of the fifty simulations. On the other hand, utility AI achieved higher task engagement at 9.74 tasks per minute while surviving 94% of the fifty simulations despite only averaging 50.14% task efficiency. This is reinforced when comparing the NABAS score for both frameworks which favours utility AI at 0.85 while behaviour trees only scored 0.58.

Utility AI demonstrated more adaptive decision-making than behaviour trees in a controlled Unreal Engine 5 scenario involving separate identical NPCs surviving fifty, five-minute simulations by managing dynamic internal variables such as health, hunger and hydration. Although this study is limited to a controlled survival scenario with a limited number of NPCs and predefined behaviours and decision variables, it provides a structured foundation for future work as well as showcasing the characteristics of both behaviour trees and utility AI in a controlled survival environment.

Table of Contents

Abstract	2
Introduction.....	4
Research and Planning.....	5
<i>Problem Identification</i>	5
Literature Review	6
<i>Behaviour Trees</i>	6
<i>Utility AI</i>	8
<i>Conclusion</i>	10
Planning	11
<i>Proposed Solution</i>	11
<i>Product Requirements</i>	11
<i>Justification of Tools</i>	12
Practical work	13
<i>Game Design</i>	13
<i>Overall Software Design</i>	15
<i>Behaviour Trees in Unreal</i>	16
<i>Behaviour Tree Code Architecture</i>	17
<i>Utility AI in Unreal Engine</i>	18
<i>Utility AI Code Architecture</i>	18
Testing	19
<i>Software Testing</i>	19
<i>Approach to Comparative Testing</i>	20
<i>Testing Metrics</i>	21
<i>Average Task Efficiency Percentage</i>	21
<i>Average Task Engagement Rate</i>	22
<i>NPC Appropriate Behaviour and Adaptability Score (NABAS)</i>	22
<i>Comparative Testing Results</i>	24
<i>Average Task Efficiency Percentage</i>	24
<i>Task Engagement Rate</i>	25
<i>NABAS Score</i>	25
Evaluation	25
<i>Results Discussion</i>	25
<i>Product Discussion</i>	26

<i>Project Process</i>	29
Conclusion and recommendations	30
<i>Conclusion</i>	30
<i>Recommendations</i>	31
References	31

Introduction

Non-player characters (NPCs) in video games are entities within the environment that are controlled by the computer, instead of the player (Warpefelt, 2016). NPCs in videogames are depicted in a variety of forms such as humanoids, animals, vehicles or mythological beasts. NPCs aim to improve the player's experience by exhibiting appropriate behaviours and adaptable decision-making (Chen, 2024).

Chen (2024) further suggests that NPC behaviour refers to a set of rules, algorithms and responses programmed into the NPC. Millington (2019, p. 293) expands this further by suggesting that the decision-making system will choose the most appropriate behaviour for the NPC to perform based on environmental circumstances.

Adaptable decision-making is crucial for improving player experience. Sprock et al (2006) defines adaptability as the ability to successfully change behaviour under changing circumstances. Dynamic circumstances can range from changes in the player-controlled character's behaviour to environmental factors. Adaptability occurs when a behaviour is displayed that was not pre-programmed to occur, instead resulting from consistent evaluation of circumstances.

Video game NPCs are controlled by a range of AI decision-making algorithms, none of which are universally agreed upon as the most optimal. Each algorithm is appropriate for distinct use cases based on the level of adaptability and behaviour intended to be displayed by the NPC (Dawe et al., 2019). When developing an NPC, it is of great value to the programmer to be conscious of the various decision-making algorithms that determine behaviour.

Because of the range of decision-making methods, it can be difficult for AI programmers to narrow down one method for NPC behaviour. The decision not only relies upon the decision-making framework itself, but the programmer's knowledge, engine choice, game genre and desired behaviour displayed by the NPC.

While decision making methods such as finite state machines (FSM), neural networks and goal-orientated action planning (G.O.A.P) offer their own unique advantages, this report investigates how behaviour trees and utility AI decision making models impact NPC behaviour and adaptability. Developed and tested in Unreal Engine 5.6 (Epic Games, 2025), a commercial game engine widely used within the industry.

Research and Planning

Problem Identification

When developing NPCs, selecting the correct decision-making framework is imperative to ensure that appropriate behaviour is displayed. Making an uninformed decision could lead to longer development times, incorrect behaviour and a negative player experience.

Unreal Engine (Epic Games, 2025) supports both behaviour trees and utility AI decision-making frameworks. However, it is important to note the large disparity between the support for these two frameworks within the engine. Unreal Engine provides developers with native behaviour tree and blackboard assets alongside built-in classes for tasks and services. Meanwhile, utility AI lacks the same level of support. Utility AI is supported through state tree selectors, where states are executed based on a utility score (Epic Games, 2025). This disparity could lead to developers defaulting to using behaviour trees due to better support and documentation. The decision to use behaviour trees without considering utility AI could lead to sub-optimal NPC behaviour and adaptability. There are most certainly situations where using an alternative approach such as utility AI could lead to more adaptable and appropriate decision-making. Clear results outlining the differences between behaviour trees and utility AI could benefit small studios and independent developers who may lack the time and resources to prototype and compare the two systems.

Despite both decision-making methods being widely adopted in game development, there is limited academic research comparing the impact on NPC behaviour and adaptability, particularly within Unreal Engine 5.

Behaviour trees and utility AI are fundamentally different by design. Behaviour trees are stateless, so behaviours are completely unaware of each other therefore, the outcome is predictable as previously executed behaviours are not a factor when determining decision-making outcomes (Dawe et al, 2019). Alternatively, utility AI uses scoring systems to determine behaviours which can be less predictable due to the dynamic nature of the system (Dawe et al. 2019). Predictable behaviours can be a double-edged sword due to higher levels of predictability being easier to expand upon and debug meanwhile, less predictability can lead to more adaptive behaviour that is harder to control. The contrasting approach of both frameworks means that the most effective

one is highly situational, mainly depending on the intended NPC behaviour, making an informed comparison necessary.

The question this project aims to answer could assist developers when developing their own NPCs. Showcasing the impact of behaviour trees and utility AI on NPC decision-making and adaptability could provide developers with clear insights into how the frameworks could impact their own NPCs decision-making logic, as well as saving development time on prototyping.

Literature Review

Behaviour Trees

Behaviour Trees became popular within the games industry following their implementation in Halo 2 (Isla, 2005). Since then, behaviour trees have become widely adopted in the industry due to their simplicity and modularity (Lim, 2010, Colledanchise, 2018). The simplistic nature of behaviour trees allows for quick prototyping and expansion as behaviours are programmed to be unaware of each other (Dawe et al, 2013, p 53). This design is contradictory to that of Utility AI due to the utility calculation factoring in other behaviours to determine appropriate decisions. Consequently, simplicity can lead to less adaptable decision-making due to decisions being executed based on a single variable unlike utility AI, which considers a range of variables.

Despite claims of simplicity, behaviour trees can become extremely complex as they grow (Barriales, 2015). Barriales (2015) also suggests that complex trees are significantly more difficult to modify as understanding the consequences of changing or adding logic is not trivial. This suggests that the implied simplicity of behaviour trees may be dependent on planning and design decisions (Francis, 2015). The importance of scalability is relevant as for an NPC to display adaptive behaviour, the corresponding behaviour tree needs to be expandable to execute any unforeseen behaviours that occur during the decision-making process.

They are composed of a hierarchical structure of nodes. The hierarchical design can be extremely powerful due to processing time not being spent on unnecessary decisions (Dill, 2011). Unreal Engine (Epic Games, 2025) support behaviour trees natively. Unreal Engine's behaviour trees execute from top to bottom and left to right. This order of execution deviates from utility AI which bases decisions on a utility score or weight. The lack of flexibility in behaviour tree decision-making can lead to less adaptable behaviour as the tree must run from the root and cycle through behaviours from left to right until an appropriate behaviour is selected (Dawe et al, 2013, p 53). This nature of execution requires the programmer to consider the order of tasks when designing the

trees hierarchy. Failure to appropriately structure the hierarchy can cause too predictable behaviour and therefore, decrease adaptability (Millington, 2019).

A vital node type of a behaviour tree is a task node. A task is an instruction such set by the programmer that will return either success, failure or running (Millington, 2019, Uludağlı, 2023). Tasks are also referred to as “leaf nodes” or “execution nodes” (Colledanchise, 2018, Uludağlı, 2023). To reach the leaf nodes of the tree, Composite nodes are used. Composite nodes are parents to leaf nodes (tasks) and control the flow of execution. Composite nodes are also referred to as “Branches” (Millington, 2019, p-334-370) or control flow nodes (Colledanchise, 2018). Composite nodes consist of three categories selector, sequence and simple parallel (Epic Games, 2025, Millington, 2019, p-334-370). Uludağlı (2023) claims selectors “Behave like an OR gate” as they will only return success when one of their children (task nodes) succeeds. While a selector’s children are failing, it will keep trying (Millington, 2019, p 334-370). On the other hand, sequence nodes “Behave like an AND gate” (Uludağlı,2023) by only succeeding when all its children succeed, and failing otherwise (Barriales, 2015). The structure of composite and task nodes is integral to promoting adaptable and appropriate decision-making. These nodes promote reactive planning (Millington, 2019) which can improve adaptability due to being able to respond to dynamic and unpredictable changes in the game. This approach to planning is identical to utility systems that also react to dynamic changes in the game to calculate a utility score (Dawe et al, 2013, p 54)

Conditions allow for behaviour trees to receive information from the world to inform appropriate decisions. (Champanandard, 2013). Unreal Engine recommends to the use of decorators to replicate the functionality of conditions. Decorators operate similarly to conditionals by querying the game world but, instead of being attached as a leaf node, they are attached to composite nodes to control the flow of execution (Barriales, 2015, Chen, 2024). Unreal Engine (Epic Games, 2025) claims that using decorators as conditionals improves readability due to all leaf nodes being tasks, it is easy to see the order of execution. Decorators can be a great tool to increase adaptability due to preventing inappropriate decisions from occurring using Boolean negation (Champanandard, 2013). Although Boolean values can improve adaptability of behaviour trees, they are still limited to either true or false. Boolean values offer significantly less context when compared to the array of considerations utility systems use to determine behaviours (Dawe et al, 2013, p 53).

Behaviour trees are commonly used in combination with a blackboard architecture to read and write data that the behaviour tree can use to make decisions (Epic Games, 2025, Barriales, 2015, Francis, 2000). This is relevant as Unreal Engine mandates the use of a blackboard architecture in their behaviour trees. This approach could prove to be an unfavourable design decision as Francis (2017) argues that a directing all data

through a blackboard architecture as a point of principle is a pitfall in behaviour tree design. Francis (2017) suggests that using ad-hoc communication instead of a blackboard may lead to increased versatility and reliability. This could mean that Unreal Engine's approach to compulsory blackboard architecture may decrease the accuracy of decision-making as the behaviour tree grows in complexity. Compulsory blackboard architecture contrasts with utility AI as although they can be used in combination, it is less common and not mandated in Unreal Engine. Since utility systems don't need a blackboard architecture within Unreal Engine, their impact on decision-making could be more accurate as communication with the game world is more direct and precise.

Ultimately, despite characteristics that suggest readability and structural simplicity and clarity, their impact on adaptive NPC behaviour adaptability heavily relies upon thoughtful design. The rigid order of execution may limit the level of adaptability achievable. This provides a useful basis for comparison with utility AI, which offers a fundamentally different approach to decision-making.

Utility AI

Utility-based approaches are commonly used in games that require complex and adaptive decision-making (Dill, 2011). Examples of AAA games that use utility AI for NPC decision making include, The Sims (Maxis, 2000-2017) and Empire Earth (Stainless Steel Studios, 2001).

Utility theory is a concept that predates video games and even computers (Graham, 2013). The foundation of utility theory is that decisions are weighed, scored and sorted based on considerations to determine the preferability of potential actions known as a utility (Dawe et al, 2013, p 54). Mark (2009) states that it is vital to note that utility is not the same as value. Value is a measurable metric such as the price of an item. On the other hand, utility is a measurement of desirability, and desirability is heavily reliant on context of the situation. This approach to decision-making differs from behaviour trees by adding more context to each decision instead of using Boolean values (Dawe et al, 2013) which can lead to less predictability without proper precautions. Therefore, utility theory may not be the best approach for creating NPCs who need to execute specific actions precise times (Dawe et al, 2013, p 55). This is fundamentally different from behaviour trees which use static, predetermined probabilities to determine actions (Dill, 2011).

The method of determining actions is commonly done in two ways. The first method is to score every action and execute the action with the highest score. The second method involves assigning a weight to every action and use a weight-based random selection to execute an action (Dill, 2011). Unreal Engine (Epic Games, 2025) adopts these approaches by allowing for child states within a state tree to be selected by either highest utility score, or random weighted by utility. Graham (2013) suggests that weight-

based random selection could occasionally cause NPC to behave unintendedly. However, Dill (2011) proposed a solution known as “dual utility” or “bucketing” which combines the two approaches. Despite solutions like bucketing improving architecture, tuning new behaviours could prove tricky. This can occur when a new behaviour is added to the array of alternative behaviours that all compete to achieve the highest utility for each decision (Dawe et al, 2013).

When calculating utility, consistency is imperative. Since comparing all utility scores is required to determine an action, they must all use the same scaling method (Graham, 2013). Unreal Engine adopts this approach when calculating a utility score with multiple considerations to create a normalised score between 0 - 1. However, normalisation only occurs when the weight is set to 1. This approach provides two methods of scoring and will provide consistent outcomes as long as one approach is used consistently (Graham, 2013).

To calculate the expected utility, a common method is to multiply the utility score by the probability of each possible outcome. The mathematical expression. Eq. 1 (Graham, 2013) represents this calculation.

$$EU = \sum_{i=1}^n D_i P_i$$

Eq. 1 Expected utility mathematical expression

The desirability of an action, represented by D, is multiplied by the probability, represented by P. The probability is normalised to ensure that the sum of all probabilities is 1 (Graham, 2013).

The expected utility is calculated for all possible actions which are then compared against one another to select the highest expected utility. This concept is known as maximum expected utility (Russel, 2002). This process is fundamentally different to behaviour trees as it uses an array of considerations to determine the most appropriate output, instead of being limited to true or false. This additional layer of context is an advantage to displaying adaptable behaviour because the outcome is typically determined by a multitude of considerations.

What makes utility potentially difficult to calculate is the concept of marginal utility (Mark, 2009). Marginal utility is integral to NPC decision-making as the desirability's of the NPC are not static and change over time (Mark, 2009). Unreal Engine and Mark (2009) provide mathematical functions to define the rate of change in a NPC's desirability's over time. Because of the lack of native support compared to behaviour trees, limited functions are provided to calculate utility and are known as “considerations”.

Utility AI can show more adaptable behaviour in non-scripted situations than behaviour trees. The concept of adaptable NPC behaviour refers to the ability to react to unanticipated events and recover dynamically from poor decisions (Lim et al, 2010). This is important as it can lead to less predictable decision-making which further leads to less repetitive behaviours. The dynamic nature of decision-making is a fundamental contrast to behaviour trees that use static decision-making based on Boolean values. However, the adaptability displayed can come at the cost of predictability. Utility also shares potential architectural issues with behaviour trees, becoming more complex as they grow but from a tuning perspective as opposed to structural.

Conclusion

Existing literature provides clear guidance on how to design and implement architecturally appropriate behaviour trees and utility AI. Incorporating recommended design decisions for either framework can lead to increased modularity and readability. Ensuring that decisions are made appropriately as well as potentially assisting in locating and debugging errors within decision-making systems. Previous research supplies a foundation of knowledge for designing efficient decision-making systems. This foundation can contribute to more accurate comparisons between behaviour trees and utility AI as the decision-making logic will not be impacted by poor architecture.

Current literature also showcases what can be expected in NPC behaviour and adaptability when controlled by both decision-making systems. It is outlined throughout that behaviour trees are more rigid and predictable meanwhile, utility AI is more flexible and dynamic. Although they are never directly compared, this contrast in the literature shows that there are clear differences on their impact on NPC behaviour and adaptability.

However, despite the characteristics of each framework being clear in individual research, previous literature does not provide a direct controlled comparison between both frameworks in a consistent game engine. This leaves a gap in the literature pertaining to understanding how both frameworks differ in terms of decision-making and adaptability under identical NPC scenarios and evaluation conditions. Because the literature suggests that behaviour trees offer predictability whilst utility AI offers flexibility, this project needs to test whether these suggestions are consistent when directly compared under identical conditions in Unreal Engine 5, by producing quantifiable distinctions in NPC behaviour and adaptability. Without direct comparison, AI programmers are shortsighted when determining the most appropriate framework for their circumstances. The investigation that follows aims to address this gap directly by providing evidence that clearly shows the difference between behaviour trees and utility AI under identical circumstances to provide a foundation for future research and AI programmers.

Planning

Proposed Solution

To explore the impact of behaviour trees and utility AI on NPC behaviour and adaptability, many approaches could be appropriate. Lim (2010) measured the effectiveness of behaviour trees and finite state machines. The comparison was carried out within an evolutionary environment for a real-time strategy game. This approach was somewhat limited due to not isolating the decision-making methods to determine effectiveness.

Alternative approaches to the problem utilised player perception. The impact of the respective decision-making framework was measured from a human perspective. While this approach provides benefits in measuring difficulty and believability, perceiving NPC adaptability could prove difficult without extensive knowledge of the inner workings of the system. This approach is not appropriate to the topic because outcomes do not consider adaptability.

More complex approaches could use mathematical data to measure how NPCs behave in specialised environments. Simonov et al (2018) developed a simulation environment in Unreal Engine to predict crowd behaviour using NPCs. This approach generates quantifiable data that could be used to compare decision-making frameworks.

The process of determining an appropriate solution resulted in the development of a prototype simulation in Unreal Engine 5. The aim of which was to create an environment for separate identical NPCs. The environment would allow for controlled experiments to measure how behaviour trees and utility AI impact NPC behaviour and adaptability by allowing for both scripted and non-scripted decision-making to be displayed, to more accurately demonstrate the differences between both frameworks. The controlled nature of experiments means that findings could be more accurate due to the isolation of each framework. Isolating the NPCs is the most appropriate method to observe behaviour tree's and utility AI's impact on decision-making as having them compete simultaneously would significantly impact findings, leading to misleading recommendations.

Product Requirements

To develop an appropriate solution, specific requirements must be met to maintain consistency during experiments. Failure to compare behaviour trees and utility AI in a controlled manner could result in misleading findings. Because of this, the following product requirements will be applied with the aim of increasing consistency during comparative testing.

NPCs will be identical aside from the decision-making framework determining their behaviour. This ensures that measurable differences in behaviour and adaptability

result from the decision-making framework being used. This is consistent with Simonov et al's (2018) controlled simulation approach.

NPCs will share the same spawn location in the same environment. This ensures that when the NPCs leave the base patrolling state to execute a task due to a change in behaviour, neither NPC has a positional advantage to complete the task.

Behaviour trees and utility AI will have identical behaviours, tasks and conditions. This establishes consistency between decision-making frameworks by minimising the variables impacting NPC behaviour and adaptability.

The only character within the environment will be the NPC. This could lead to a more a more consistent and accurate testing environment due to minimising number of external factors impacting the environment.

From a non-functional standpoint, a stable framerate is necessary to ensure that the decision-making process is not impacted by frame drops or skips, which could cause inconstant results.

Data gathered during testing should also be contextual, readable and recordable to ensure that metric comparisons are consistent. This approach should ensure that the metric data gathered during testing is reliable and consistent.

To reinforce the validity of data gathered from experiments, the simulation must produce consistent outputs across a multitude of runs. Without repeatability anomalies in the data could be misinterpreted to be caused by the respective decision-making framework.

Justification of Tools

Unreal Engine 5 offers great tools for building an environment suitable for comparing NPC behaviour and adaptability. Unreal Engine features a built-in behaviour tree asset that contains the necessary nodes outlined by Millington (2019), Colledanchise (2019) and Barriaes (2015). Behaviour trees in Unreal Engine are event driven; this is an optimisation over first generation behaviour trees due to listening and responding to events instead of traversing the tree every frame (Champandard, 2013). However, Unreal Engines behaviour trees mandate the use of a blackboard architecture to read and write data. Francis (2017) argues that this method of communication can be unnecessary and lead to suboptimal outcomes.

Unreal Engine also features native Utility AI support within state tree assets. In this case, the state tree contains the task logic and data while Utility AI determines behaviours based on the stored data. Unreal supports the commonly used methods of determining a utility score outlined by Dill (2011) but does not include support for dual utility or bucketing. Although utility AI does not have its own asset in the engine, it is

important to note that when set up correctly within the state tree, utility AI will act as the only factor when determining NPC behaviour.

Unity engine (Unity Technologies, 2015) was also taken into consideration due to its popularity within the games industry. But, since Unity does not natively support either AI framework, they would have to be created from scratch. This could lead to less informed recommendations due to a disparity in the design of the decision-making frameworks. Using Unreal Engine's natively supported decision-making systems could lead to less disparity in further research due to the design of the frameworks being consistent across future literature. Additionally, using pre-designed frameworks allows for more emphasis to be placed on creating an environment that encourages adaptability, with the intention of gaining more comprehensive results during comparative testing.

Practical work

Game Design

To appropriately compare both NPC decision-making methods, it was important to make informed and appropriate game design decisions. Creating an environment where one decision-making method has a clear advantage could produce misinformed evidence. To create an environment to appropriately compare the two methods, the focus of the game should be the NPC itself.

Academic research in the subject used simulations of NPCs behaving like pedestrians to research potential risks of overcrowding in public spaces (Simonov et al, 2018). While the outcome of this research was to produce an alternative approach to pedestrian simulations, it demonstrated valuable results in NPC behaviour in the presented scenario. A commercial example of game design centered around AI controlled NPCs is seen in The Sims franchise (Maxis, 2000 - 2017). This franchise is widely recognised example of AI driven design. The world in Sims is populated with NPCs that adapt constantly to the player's decisions and the environment.

This approach should provide the best results when comparing behaviour trees and utility AI and their impact on NPC behaviour and adaptability. By creating an environment around the NPC, there will be minimal interference from the player-controlled character. This approach aims to improve NPC decision-making adaptability and consistency during comparative testing.

To minimise player-controlled character interference, the pawn was set to have no collisions. This was done to ensure that the NPC to take the most optimal path to reach its destination without colliding with the player, leading to less accurate task response times. Additionally, if the player were to collect a resource by overlapping with the object's collision box, NPC decision-making will be impacted due to a non-controlled

variable interfering with the environment. While this could prove claims of adaptability, replicating this behaviour consistently throughout testing would be incredibly difficult and most likely result in inaccurate findings.

The environment is designed to create a survival simulation for an NPC character. A survival scenario was chosen as it encourages both pre-determined and adaptable behaviour to stay alive. When compared to other approaches (Simonov et al, 2018) the survival scenario encourages more of a balance between scripted and non-scripted situations while maintaining the isolation of the NPC. Isolating the NPC was imperative to the design, as using a player-controlled character could impact NPC decision-making based on the user's skill level and comfortability with games. Therefore, a survival scenario provides a solid foundation for future literature because it will be simple to replicate.

The survival environment for this project consists of a persistent set of locations for food, water, sleeping, shade and warmth. These were chosen to be the dedicated survival mechanics for the NPC as the objective of the simulation was for the NPC to stay alive. The NPC would stay alive by executing behaviours based on internal variables: hunger, hydration, tiredness and temperature. These variables were selected as they would provide sufficient needs for both predictable and adaptable decision-making as occasionally, the NPC will have two competing needs that require prioritisation. This approach aims to showcase the strengths of both decision-making frameworks as behaviour trees should execute tasks with the highest pre-determined priority regardless of other behaviours (Dawe et al, 2019). Meanwhile, utility AI should demonstrate more adaptable behaviour due to reacting to unscripted events such as a change in temperature to redetermine decision-making based on a dynamic utility score (Graham, 2013).

To further encourage more NPC adaptability, the resources required to satisfy the needs of the NPC will spawn on consistent timers. This approach promotes adaptability as occasionally; the NPC will not be able to execute the highest priority behaviour and will need to adapt to the circumstances to maintain survival.

The only variable that determines which location or object the NPC moves to and uses is the decision-making method controlling their behaviour. This environment design focuses on emphasising the differences between behaviour trees and utility AI by isolating behaviours to only be determined by the dedicated decision-making method. This design could provide more accurate results due to minimal interference from unintended sources.

Overall Software Design

To create an environment in which AI NPC's can be compared, it was imperative that an appropriate program architecture was built to support and encourage a fair comparison. My approach was to use Object-Oriented Programming (OOP) to structure the codebase. OOP supported a fair comparison between both AI approaches by minimising the number of variables that determine NPC behaviour. Encapsulation and inheritance allow for both NPCs to inherit base functionality from a base class therefore, improving consistency and supporting a fair comparison as the NPCs will have identical logic.

The NPCs base class stored the variables pertaining to the NPCs health, hunger, hydration, tiredness and temperature. Inheritance ensured that both child NPC classes contained the same variables and methods set in the base class and therefore, shared identical base functionality. Encapsulation and Inheritance provided increased experimental availability as both NPCs share identical functionality, the decision-making system was isolated as the independent variable.

Encapsulating the variables that determine behaviour within the NPC base class can improve consistency across decision-making processes. This approach involved setting Boolean variables to true when specific conditions are met, such a starving Boolean to true when the hunger value is equal to or below a set threshold. Encapsulating this logic in the base class is a cleaner and more maintainable approach than individually setting Booleans in the task code. This helps to ensure that the consistency related requirements are met by reducing the chance of behaviour determining variables being different in each decision-making framework.

Unreal Engine has two methods of programming, a visual scripting system called blueprints and C++. All projects created in Unreal have access to both methods of programming and typically combine them as to create an instance of a C++ class a blueprint based on that object must be made. For this project, a combination of the two methods was used. C++ was be used to develop classes that contain a significant amount of logic as it can be more performant when compared to creating large classes in blueprints. Additionally, C++ improves traceability of errors during testing as UE_LOG provides detailed information about what logic is executing at runtime, allowing for data monitoring and logging potential issues (Epic Games, 2025). Another advantage of C++ is the ability to use data breakpoints to quickly determine the root cause of an error by isolating the data from its address in memory (Shao, 2025).

Casting in C++ can be less computationally expensive than in blueprints as C++ classes in Unreal Engine are significantly smaller in memory size as opposed to blueprints and therefore each cast requires less CPU power and have little impact on performance which is important as an inconsistent frame rate could impact the behaviour of the

NPCs leading to misleading findings. A stable frame rate is a requirement for the testing of this project as it ensures that behavioural outcomes are attributable to decision logic rather than runtime variability.

Behaviour Trees in Unreal

Unreal Engine natively supports behaviour trees in the editor and in documentation. The asset is packaged with the necessary node types to build a complex behaviour tree. Because of native support, the architecture of the tree does not need to be programmed which could increase consistency across future literature as the background processes for tree nodes will be identical. This means that Unreal Engine's behaviour trees are a solid option for demonstrating NPC decision-making and adaptability as well as providing a foundation for future literature to build upon.

Unreal Engine's behaviour tree asset was designed in conjunction with blueprint graphs to make the user interface more readable and familiar to those who use blueprints. The asset includes the necessary nodes to develop a functional behaviour tree. Additionally; a blackboard asset is also provided within the engine which operates as the brain of the behaviour tree to make decisions based on user defined keys.

Unreal Engines behaviour trees are event driven as opposed to traditional behaviour trees that check for updates every frame. This difference can lead to better performance and ease of debugging as unnecessary iterations are not executed because they will not take place without an event and therefore, only relevant executions are shown. (Epic Games, 2025). The event driven nature of Unreal Engines behaviour trees ensures that state transitions are consistent and predictable and will provide more accurate results during comparative testing.

Conditionals are also implemented differently to traditional behaviour trees in Unreal. As opposed to using task leaf nodes whose sole purpose is to succeed or fail, Unreal recommends the use of decorators. Decorators serve the same purpose as task leaf nodes but provide benefits to behaviour tree readability and intuitiveness (Epic Games, 2025). The benefits are achieved by the decorator attaching itself to a sequence node to control the flow of execution of the node's children based on a selected blackboard key. The benefit of this implementation is that observer aborts are easier to read and implement as the condition to abort the task early is set on the sequence node instead of the task itself. Unreal Engine's implementation of conditionals ensured that tasks are executed consistently and predictably, minimising the margin of error therefore providing more accurate results during testing.

Behaviour Tree Code Architecture

The core software design principles mentioned previously were continued to develop behaviour tree task and service nodes. To implement this approach effectively, a specialised base class for all behaviour tree tasks and services nodes was created, inheriting from Unreal Engines own base classes. Within the specialised base classes, polymorphism was used to override specific methods from Unreal Engine's base classes to implement dedicated logic. For example, the base behaviour tree task class overrides the public Execute Task and Tick Task methods.

To appropriately make use of inheritance, pointers to the NPC, AI Controller and Blackboard Component were declared as protected member variables, so the child classes would share access. Within the function definitions for the overridden methods, the protected pointers are initialised with safety countermeasures. The subclasses of the specialised base class use inheritance to obtain all non-private members, providing access to the logic within the declared functions. Further specialisation was implemented within the subclasses by overriding the necessary methods and using polymorphism to create distinct logic pertaining to the intended outcome of the class. The same approach was applied to the base behaviour tree service class but instead, overriding the TickNode and OnBecomeRelevant methods from Unreal Engine's base service class.

This approach significantly reduced the amount of code within each subclass as casting to the necessary classes was managed by the superclass. Inheriting access to the NPC and AI controller classes improved safety as only one cast is executed by the base class rather than each subclass individually casting to these objects. Furthermore, the logic within the subclasses was very specific to the function of the class resulting in improved readability and smoother debugging if errors occurred. The reduction in errors from null or misdirected pointers ensured that testing the NPC controlled by the behaviour tree was consistent as all subclasses share the inherited pointer to that specific NPC and its controller.

The decision to approach to code architecture in this manner for behaviour trees was to promote scalability. Francis (2015) emphasised the importance of scalability to minimise errors when expanding. To effectively illustrate adaptive decision making, there must be a range of behaviours which can lead to more complex trees where expandability it imperative so that new behaviours do not impact previous ones. This concept is also outlined by Dawe et al (2013) who states that behaviours should be programmed to be unaware of each other. OOP made this concept uncomplicated as the subclasses code was significantly more readable, making the purpose of the task easier to understand therefore, reducing the chance of dependencies.

Utility AI in Unreal Engine

Like behaviour trees, Unreal Engine supports utility AI. Utility AI is supported through the state tree asset which includes options to select behaviours based on a utility score or weight. It is important to note that in Unreal Engine, the state tree is a framework to hold the behaviours and related tasks. Since the state tree uses utility AI as the decision-making algorithm, the only variable impacting NPC decision-making and adaptability is the utility score. This means that utility AI in Unreal is a solid option for demonstrating NPC decision-making and adaptability as well as providing a foundation for future literature to build upon.

To use utility AI in the state tree, the corresponding selection behaviour must be set in the root. The two utility selection behaviours are “Try Select Children With Highest Utility” and “Try Select Children At Random Weighted By Utility” Graham (2013) suggests that the latter selection behaviour could occasionally produce unintended behaviour. Therefore, selecting the child with the highest utility was the chosen approach for this project.

Unreal also supports methods of calculating a utility score. The state tree utility selectors use utility considerations to determine the utility of a behaviour. Considerations can be of three types: Constant, Enum and Float. Multiple considerations can be used to calculate the maximum expected utility (Russel, 2002). The most important of these considerations for this project is the float input because the variables that determine NPC behaviours being of type float. This allowed for the identical values being used to determine NPC behaviour. However, utility AI uses response curves to change the utility score based on the input float value, in contrast to behaviour trees that use Boolean values. This can allow for more adaptable behaviour as for example, the hungry state doesn't require the NPC to be starving to occur. However, since it is a requirement to remain consistent between decision-making methods the starving value is identical between decision-making methods meaning that if the NPC's hunger value is less than or equal to the starvation value the utility becomes 1 therefore, making it the highest scoring outcome.

A weight input is also provided to give additional bias for or against its child behaviours on top of the considerations, but the final score will not be normalised (Epic Games, 2025). Based on this, all weight inputs for behaviours will be set to 1 to remain consistent with previous literature that uses Eq.1 to calculate utility resulting in a normalised score between 0-1.

Utility AI Code Architecture

As a result of the base Unreal Engine code for state tree tasks, it would have been inefficient to create a base class for all state tree tasks. This is due to each task requiring unique instance data. Instance data is a struct containing the necessary input

data and output data such as the NPC class whose AI controller owns the state tree component. Instance data structs behave like a blackboard as it writes data to the state tree which is read and factored into the utility calculations. Francis (2017) argued that Unreal Engine's state tree's style of ad-hoc communication could lead to improved versatility and reliability.

Unreal Engine uses base structs to define state tree task logic. Within these structs, the necessary methods are defined as virtual, intended to be overridden when inherited. For example, each state tree task overrides the Enter State and Tick methods. Within inherited tasks, the unique instance data must also be initialised by overriding the GetInstanceDataType struct to return the unique instance data of that class. The difference between behaviour trees using classes and state trees using structs is negligible due to the only discrepancy being structs are public by default. This is extremely unlikely to impact decision-making or adaptability.

To properly replicate the functionality of a behaviour tree within a state tree, global tasks can be implemented. Global tasks serve a similar purpose to behaviour tree services in that they query the game world every frame to check if conditions are met. The logic in the global state tree tasks is more simple than regular tasks, only checking Boolean variables set in the base NPC class.

Despite not using OOP to create a custom base state tree task struct, OOP principles were still used within the task logic. Because the utility NPC class inherits from the base NPC class, the same variables were used in the state tree task logic to determine decision-making. This satisfies the consistency requirement stating that both decision-making factors as the same variables were implemented into behaviour tree services to determine NPC behaviour.

Testing

Software Testing

Before comparative testing occurred, it was important to ensure that the simulation worked correctly, without any issues that would impact results. This is an important step because locating and fixing bugs in the code ensures that both decision-making frameworks work as programmed as well as insuring consistency.

Most often, game software requires a different approach to testing than conventional software emphasising aspects such as game mechanics over security or maintainability (Kasurinen and Smolander, 2014). Based on this, the approach to testing the software was from a functionality standpoint.

To test the functionality of the product, a white-box technique was applied. This technique enabled more in-depth verification of the decision-making frameworks working as intended. The white-box technique can lead to improved consistency and predictability in NPC decision-making due to testing occurring with an understanding of the internal systems impacting decision-making.

The process of testing would typically start with isolating the newer code. For example, when adding a new task to either decision-making framework, it would first be tested independently of other tasks. After verifying the task logic worked as intended, it would be incorporated into the larger framework, and testing would be carried out on the decision-making system to ensure that the new task behaved as intended as part of the larger system. This approach allowed for easier problem identification by isolating features before incorporating them to ensure proper functionality.

Unreal engine's UE_LOG was a vital part of the testing process, allowing for more accurate problem identification. Throughout testing, UE_LOGs would be added to the code to ensure that the correct data was being used to execute logic. This process helped safeguard against incorrect data being used to inform decision making which was expected to occur due to the number of variables being used to determine decision making.

The testing process was heavy focused on ensuring that the decision-making frameworks powering the decision-making process behaved as intended. Because of this focus, the decision-making process was robust and therefore, could provide accurate results when comparing behaviour trees and utility AI's impact of NPC decision-making and adaptability.

Approach to Comparative Testing

To conduct fair comparative testing between both decision-making frameworks, fifty simulations will be carried out. A simulation refers to a five-minute session where the NPC must survive by carrying out appropriate and adaptable behaviour.

Ideally, a higher quantity of simulations would be undertaken but, due to time and hardware limitations, fifty was chosen as an appropriate and quantitative number. The five-minute simulation length was chosen to provide an appropriate length of time for adaptable decision-making to occur due to unexpected events such as a dramatic change in temperature taking place to encourage adaptable decision making. Despite this, it was still possible for an appropriate change in temperature not to occur which could lead to gaps in data. Furthermore, there were situations where the NPC may not have survived long enough to experience a large change in temperature however, this can be viewed as a lack of appropriate or adaptable decision-making because the NPC could not survive the dedicated time.

To gather quantifiable data to effectively compare behaviour trees and utility AI, metrics were gathered after each simulation. The metrics were influenced by previous literature such as Millington (2019) suggesting that behaviour trees could lead to more predictable NPC behaviour and utility AI potentially causing less predictable but more adaptable behaviour (Dawe et al, 2013, p 55). Based on these claims, the following metrics were gathered to compare both decision-making frameworks.

Testing Metrics

Average Task Efficiency Percentage

Task efficiency refers to the number of tasks completed that were premeditated. For example, if the NPC executes the task to eat food while starving, the task was necessary and would be considered efficient. The logic behind tracking this metric involved storing the corresponding value at the start of the task execution function and comparing the stored value with the corresponding value that would require the task to be executed. For example, when the EatFood function is called by either decision-making framework, the NPC's current hunger value would be stored locally, after the eating logic is executed, the stored value is compared with the starvation value to check if the stored value is less than or equal to the starvation value. If true, a statistic variable pertaining to that specific task would be incremented.

Task efficiency aims to capture the predictability of decision-making by defining an action carried out by the NPC as necessary. A necessary action is an action that is pre-determined and therefore, can be expected. This aims to demonstrate how accurately each decision-making framework responds to pre-determined behaviours to provide a foundation for future developers programming NPCs with heavily scripted behaviours that need to be executed with accuracy.

Task efficiency is limited by how it demonstrates adaptability. Since adaptability is a main component in the research of this project, task efficiency is an obscure method of showing differences in adaptability. Lower task efficiency correlates to more adaptive behaviour as it indicates more independent decision-making. Furthermore, task efficiency was not an appropriate determinator of other metrics. It was difficult to find correlations between task efficiency and other evidence of appropriate behaviour.

From an implementation perspective, the process of calculating the task efficiency could be improved as it does not consider the full decision-making process to determine if the task was necessary. The process is contained within tasks that finalise the decision-making process. This means that the tasks executed beforehand to reach the final decision are not considered. The reason for this approach was to improve the consistency of the data by implementing the calculation in the corresponding method within the NPC base class that would be called by the decision-making framework.

Despite, potential improvements, the data met expectations and provided a clear difference between behaviour trees and utility AI.

Upon the end of the five-minute period or NPC death, the task efficiency percentage for that simulation would be calculated using the following mathematical equation.

$$\text{Task Efficiency} = \text{NecessaryTasks} \div \text{Total Tasks} \times 100$$

Eq.2. Task Efficiency mathematical equation

This metric aims to align with the claim that behaviour trees offer more predictable behaviour by averaging a higher task efficiency percentage than utility AI. On the contrary, a lower task efficiency could show more adaptability as the NPC is more autonomous in its decision-making.

Average Task Engagement Rate

Task engagement refers to the number of tasks completed per minute survived. A task completed refers to task carried out that achieved an intended result such as, eating food to regain hunger. The logic behind tracking this metric involved measuring the total tasks completed and dividing the value by the time survived in minutes. A completed tasks refers to a task carried out that achieved the intended goal for example, if the NPC eats food to increase its health value above the satiation value, it would be considered a completed task regardless of whether the NPC needed to eat beforehand. This resulted in a tasks per minute metric that indicates how active the NPC was during the simulation.

The task engagement rate aims to align with the claim that utility AI can offer more adaptive NPC behaviour by displaying higher task engagement rates. On the other hand, behaviour trees should display a lower task engagement rate due to claims of less adaptability when compared to utility AI.

After the fifty simulations were conducted, the average task engagement rate was calculated using the following mathematical equation.

$$\text{Task Engagement Rate} = \text{Average}(\text{Total Tasks} \div \text{Total Time Survived} \div 60)$$

Eq.3. Average Task Engagement Rate mathematical equation.

NPC Appropriate Behaviour and Adaptability Score (NABAS)

NABAS aims to incorporate all the gathered data into a normalised score between 0-1 to demonstrate which decision-making framework has the highest impact on NPC behaviour and adaptability. NABAS is a combination of the following metrics:

Survival outcome – Whether or not the NPC survived the five-minute simulation.

Time survived – How long the NPC survived in seconds normalised between 0-1.

Need diversity – A score outlining the attention given to each need normalised between 0-1.

Response speed – Response speed to each task based on the minimum and maximum response speed for the corresponding AI type normalised between 0-1.

Task efficiency – Task efficiency expressed as a normalised value between 0-1.

The importance of each metric will be determined by its importance under the simulation's circumstances. The main goal of the simulation was to survive; therefore, survival outcome will be weighted at thirty-five percent and the time survived will be weighted at twenty-five percent. These numbers were chosen as they are a result of how the decision-making framework responded to the aims of the simulation. The decision-making framework had the most impact on NPC decision-making and adaptability should also demonstrate higher survivability and time survived.

Need diversity will be weighted at twenty percent. This number was chosen as need diversity is a key factor in determining the adaptability of each NPC. Adaptability is a main factor in what this project aims to investigate therefore, valuing need diversity at twenty percent means that NABAS will use adaptability as a key consideration when determining the overall score. It is still weighted lower than survival orientated metrics due to its specific focus on adaptability meanwhile, survival outcome and time survived relate to both adaptability and the encompassing decision-making process.

The final twenty percent is split evenly between response speed and task efficiency. Valuing both these metrics at ten percent respectively was caused by their lack of overall impact in determining NPC decision-making and adaptability when compared to the previous metrics. Response speed shows the ability to prioritise tasks effectively based on the circumstances. While it could be argued that this shows adaptability, too many uncontrolled variables impact the outcome therefore, it was determined that it was less important than previous metrics in showcasing NPC decision-making and adaptability. Task efficiency shows the predictability of decisions made in response to pre-determined events. Again, this could have been a valuable metric in demonstrating the impact of each framework on NPC decision-making but, throughout testing it was shown to have no correlation with survival outcome or time survived which hold the highest values in calculating NABAS. Therefore, it would be unreasonable to value task efficiency significantly higher due to its smaller impact on NPC decision-making and adaptability.

Although NABAS could provide a solid metric for comparison, it could be further improved by being less context dependent. NABAS prioritised survival outcome and

time survived as factors for determining a score which may prove difficult to replicate in future literature if applied outside of a survival scenario. This could be improved by incorporating fewer specific metrics that can be replicated by focusing more on the decision-making process. This would also result in necessary changes to the weighting of the factors that determine the NABAS score to appropriately integrate new metrics.

Comparative Testing Results

This section presents the findings for the previously mentioned testing methods. Data was collected from fifty, five minute simulations within the same environment adhering to the product requirements.

Average Task Efficiency Percentage

AI Type	Task Efficiency(%)
Utility AI	50.14%
Behaviour Tree	96.49%

Table 1: Average Task Efficiency Percentage

Table 1 shows the average task efficiency of the respective NPC decision-making framework. This data reflects the claims that behaviour trees offer predictable decision-making by demonstrating an almost 100% task efficiency. This shows that behaviour trees could be a better approach to creating NPCs with heavily scripted behaviours that require precise execution. Meanwhile, Utility AI scored at 50% which aligns with claims of less predictable behaviour. The lower score is most likely caused by the framework behind utility AI utilising dynamic scoring to determine behaviour instead of pre-determined scores.

However, the most intriguing aspect of this metric is its lack of connection to survival outcome as despite utility AI's significantly lower efficiency, the corresponding NPC survived 94% of the simulations. On the other hand, the NPC controlled by a behaviour tree survived 32% of the simulations meaning that task efficiency is an ineffective metric in terms of survival which is why its weighting for the NABAS score was low. The lack of connection inadvertently showcases the adaptability of utility AI as it demonstrates the ability to complete tasks autonomously without scripted events occurring.

Task Engagement Rate

AI Type	Tasks Per Minute (TPM)
Utility AI	9.74
Behaviour Tree	7.04

Table 2: Average Task Engagement Rate

Table 2 shows how the average tasks per minute (TPM) of each decision-making framework. This data shows a 32% difference between activity levels with utility AI scoring significantly higher. This data reflects claims of adaptability as a higher TPM demonstrated the ability to adapt to unscripted circumstances and by completing the corresponding task. This shows that utility AI could be a better approach to creating more autonomous and active NPCs that need to constantly adapt to changes in the environment. Meanwhile, behaviour trees scored 2.7 TPM lower than utility which showcases less engagement and adaptability. This lower score is worsened when considering that the behaviour tree NPC only survived an average of 225 seconds compared to utility AI's 294 seconds which provides behaviour trees an advantage when calculating the TPM.

When analysed alongside task efficiency, some interesting results occur. It indicates that behaviour trees have less cognitive load through less TPM however, they are doing it correctly by demonstrating a higher task efficiency. Meanwhile, utility AI's cognitive load is significantly higher shown by higher TPM, survival outcome and survival time despite less accuracy.

NABAS Score

AI Type	Average NABAS Score
Utility AI	0.85
Behaviour Tree	0.58

Table 3: Average NABAS Score

Table 3 shows the average NABAS for each decision-making framework across fifty simulations. This data shows a 37% difference between average NABAS with utility AI scoring significantly higher. This data reflects claims of utility AI displaying more adaptable behaviour than behaviour trees due to NABAS considering a range of metrics to determine a normalised score between 0-1. Since NABAS is the encompassing metric, it could be determined that utility AI has a higher impact on NPC behaviour and adaptability. On the contrary, behaviour trees scored an average of 0.27 NABAS lower than utility AI which showcases that behaviour trees have less of an impact on NPC decision-making and adaptability.

Evaluation

Results Discussion

Results show a clear difference between how behaviour trees and utility AI impact NPC decision-making and adaptability despite taking a different approach compared to previous

research (Simonov et al, 2018). Behaviour trees achieved significantly higher task efficiency 96.49% compared to utility AI's 50.14%, corresponding with claims made in previous literature (Millington, 2019; Dawe et al, 2013). However, the difference in task efficiency did not translate into more successful outcomes as behaviour trees only survived 32% of the tested simulations.

Meanwhile, utility AI survived 94% of the tested simulations while averaging 9.74 TPM despite lower task efficiency. This suggests that utility AI is more impactful on NPC adaptability than decision accuracy, allowing NPCs to respond more effectively to dynamic and unscripted situations. These findings align with claims made in previous literature (Lim et al, 2010)

When analysed together, results indicate that behaviour trees excel in controlled and predictable conditions, while utility AI provides a greater impact on NPC decision-making and adaptability in dynamic environments.

Despite evident differences in results, these should be interpreted within the constraints of the scenario rather than inadequate data collection. Because of minimal prior research into directly comparing behaviour trees and utility AI, standardised metrics are not outlined therefore, metrics were derived from claims made in prior research. This resulted in metrics focusing on the characteristics of each framework, such as adaptability and predictability. While this approach may emphasise certain behaviours over others, it provides a structured and justifiable basis for comparison, supporting meaningful conclusions within a constrained scenario.

As well as the characterised metrics gathered, survival metrics were also accumulated based on the context of the simulation requiring the NPC to survive the five-minutes to be considered successful. These metrics could be seen as flawed because of their lack of reproducibility in future work within the subject. Because of the variety of NPCs used in video games, all with different goals and objectives, it could be seen as impractical to determine which decision-making framework impacts NPC behaviour using survival-specific metrics.

NABAS was created by combining both survival and specialised metrics to produce a normalised score to serve as the main metric to determine which decision-making framework had the most impact on NPC behaviour and adaptability. It could be considered inapplicable to use survival metrics to determine NABAS due to their lack of reproducibility. However, the main purpose of the NPC in this context is to survive the five-minute period by completing tasks determined by their respective decision-making framework. This means that although survival metrics lack reproducibility, they serve as an accurate and concise metric to determine how effective the respective decision-making framework was on NPC behaviour and adaptability in the given scenario.

Product Discussion

From a technical perspective, the product satisfied most, but not all the requirements. The aim of the product was to compare behaviour trees and utility AI in Unreal Engine 5.

To achieve this goal, specific requirements had to be met to ensure a fair and consistent comparison.

Therefore, consistency was a strong requirement from both a coding and testing perspective. This was met by using OOP design when writing the NPC base class. Inheriting from the base NPC class meant that both frameworks would call the same functions and variables to determine the decision-making process. This process was continued when gathering metrics by attaching a custom metric component to the base NPC class to handle data collection. However, a discrepancy can be seen when tracking task entry time data. Task entry time was logged through a task implemented at the beginning of each state in both frameworks. Meanwhile other metrics were implemented as part of the relative task's logic, instead of having their own specialised task with the purpose of data collection. Although this discrepancy most likely did not impact data collection, an improvement would be tracking task entry data in the respective task that runs on state entry. This would improve the consistency of data collection by ensuring that the methods used for data collection is done using the same method.

OOP principles were continued when implementing behaviour tree tasks and services. Not only did this approach improve consistency through inheritance, but it also allowed for more straightforward debugging and improved code readability. The main pitfall in the behaviour implementation stems from incorporating new behaviours with existing ones. As the complexity of the tree grew, it became increasingly difficult to incorporate new behaviours. Although this was expected (Barriales, 2015), it occurred sooner than anticipated. This issue could have become worse if the project grew in scope, to prevent this, an approach could be implementing "hints" (Barriales, 2015). Hints aim to abstract decision-making by sending information to the main behaviour tree from a higher-level source to produce alternative behaviours. Not only would this approach improve the process of incorporating new behaviours in the long term but, also improve the level of adaptability shown by the behaviour tree NPC. Improved levels of adaptability would benefit this project as results show significant differences in adaptability, with behaviour trees averaging 2.70 less tasks per minute and only surviving 32% of simulations.

OOP could not be continued when developing the utility system due to limitations in Unreal Engine's state tree backend code. Despite this, the decision-making process was not impacted as within tasks, functions and variables on the base NPC class could still be called to determine decisions which aligns with the consistency requirements. Utility AI performed significantly better during comparative testing, surviving 96% of simulations with an average of 9.74 tasks per minute. This shows that the implementation of utility AI was effective due to corresponding with claims outlined in previous literature pertaining to adaptability (Lim et al, 2010). Nevertheless, similar

design issues occurred when implementing and tuning utility scores. Because the fundamental difference behind choosing an appropriate decision differs between both frameworks, maintaining consistency between the prioritisation of behaviours became difficult. The implementation in this project uses only one variable to determine a utility score which could be improved by adding more variables to ensure that the most appropriate behaviour is selected. This could lead to utility AI demonstrating more task efficiency as it scored 46% lower than behaviour trees.

When evaluating the implementation of both frameworks, clear differences can be seen. The structure of behaviour trees is fundamentally different to utility AI in terms of flexibility. This means that the behaviour tree implementation was more fragile as implementing new behaviours could result in unintended consequences to previous behaviours. Meanwhile, implementing a new behaviour to the utility system was straightforward but the complexity arose when determining the factors used to calculate a utility score. These characteristics are evident during testing as the rigid nature of behaviour trees resulted in a 96% average task efficiency showing their predictability. On the other hand, utility AI demonstrates its flexibility through high adaptability, averaging 9.74 TPM and an average .85 NABAS score.

There are noticeable limitations in the implementation of the project. Most notably are the limited number of simulations. More tests could further ensure that the validity requirement was met by decreasing the impact of anomalies in the data. Another limitation is the NPC behaviours could be considered simplistic, only featuring five states, limited mainly by time constraints. With more time, more complex behaviours could be implemented which could further show the impact of the respective decision-making frameworks. One final limitation is limiting testing to one scenario. The scenario of the project was NPC survival by fulfilling basic needs such as hunger and dehydration. This limits the results to be more relevant to a specific type of NPC. An improvement would be testing in more gameplay situations such as combat or companion NPCs.

Alternative approaches could lead to improvements within the current project. The most intriguing of which is combining the two frameworks using a behaviour tree as the structure and utility scoring within the behaviour tree services. This would be an interesting approach because utility scoring contrasts the rigidity of behaviour trees. Although performance was not an issue, utility scoring could have been optimised by caching values instead of re-calculating every tick, this could come at the expense of decreasing adaptability but would increase consistency with the functionality of behaviour tree decision-making. One final approach would be sending data to the utility system through a blackboard architecture. Because blackboards are native to the engine, it would have been a simple approach to implement. But, since previous

literature did not outline using blackboards to communicate with utility, this approach was not taken.

Project Process

The project process partially followed the plan outlined in the TOR as it served as the best-case plan for the entire project. The TOR did not account for unforeseen issues which caused changes in the project. This lack of foresight meant that the project did not reach its full potential caused from less-than-ideal adherence to the timeline outlined in the project plan. Early on, too much time was spent determining the game engine that would be used to carry out the experiment. Although the TOR states the project will be carried out in Unreal Engine 5, early prototypes were made in both Unreal Engine and Unity Engine 6.2 (Unity Technologies, 2025) to ensure that the most appropriate engine for the project's purpose was chosen. A quicker decision in engine choice could have resulted in a longer development time which could have led to more refined behaviours or more overall behaviours for both decision-making frameworks which could improve the accuracy of results.

Throughout the project, numerous skills were refined. The most notable skill that was improved was programming, caused by the implementation heavy nature of the project. An emphasis on the structure of the codebase was a focus early on, based on previous programming projects becoming increasingly difficult to maintain at scale. I learned how to properly plan and design a larger codebase by abstracting classes and utilising inheritance to prevent re-writing code to improve readability, clarity and maintainability. If the project were to be completed again, I would place more emphasis on determining comparative metrics so that the decision-making frameworks can be built around the metrics instead of them being applied in the later stages of creation.

Although unforeseen implications caused occasional deviations from the project plan's timeline, extra time would be spent resolving bugs within the code to ensure that I did not fall behind schedule. In retrospect, more foresight should have been applied to encountering bugs within the project plan. If this were a consideration, the development phase in the project timeline would also be extended to correspond with the actual development time.

The largest deviation from the TOR occurred during the experimentation phase, the metrics outlined in the TOR were altered to more accurately showcase the differences between the frameworks in the context of the game environment. This deviation was caused from poor game design vision. The TOR mainly focused on the project question and code implementation, neglecting game design which resulted in the need for more

specialised metrics to properly show the differences between behaviour trees and utility AI.

Conclusion and recommendations

Conclusion

This project aimed to investigate how do behaviour trees and utility AI impact NPC decision-making and adaptability. Previous literature established clear expectations ahead of this investigation. Behaviour trees were characterised as rigid and predictable and therefore, more suited to heavily scripted environments. Meanwhile, utility AI was identified as more flexible and dynamic in its decision-making, capable of producing highly adaptive behaviour in changes circumstances. These expectations provided a foundation for which the findings of this project could be evaluated.

To carry out this investigation, a controlled survival simulation was designed, developed and tested in Unreal Engine 5. The resulting product allowed for a direct comparison between both frameworks by implementing identical NPCs operating under their respective decision-making framework within the same environment.

The findings indicate that Utility AI demonstrated greater adaptability in non-scripted situations, while behaviour trees produced more consistent but less flexible decision-making. This was evidenced through comparative testing which produced metrics such as survivability, task efficiency and task engagement. Although behaviour trees demonstrated reliable and predictable behaviour, utility AI showed a stronger ability to adapt to dynamic situations, suggesting greater levels of adaptability.

The project achieved the main goal of analysing the relative effectiveness of both behaviour trees and utility AI. The developed product served as a consistent testing environment where NPC decision-making and adaptability could be analysed, with the decision-making system controlling NPC behaviour acting as the independent variable. Despite deviations from the original metrics outlined in the TOR, the alternative, more specialised approach still provided clear and meaningful results for comparison.

However, several limitations must be acknowledged. Although the simulation environment was sufficient for this investigation, it represents a limited model of larger scale games and therefore, may not capture the complex nature of fully-fledged games with other systems and multiple NPCs. Limiting the investigation to only one engine also confines results to Unreal Engine, meaning the results may not be representative of the project question when applied in alternative game engines.

Recommendations

Future research should expand the scope of this project, using more complex environments to encourage more adaptive behaviour while maintaining the need for predictable decisions. Future research should not be limited to the survival scenario, investigating within other scenarios could demonstrate a more objective conclusion to the research question. Expanding further, multiple NPCs could be tested concurrently within the same environment under a competitive circumstance.

Another recommendation could involve future work incorporating hybrid approaches, combining behaviour trees and utility AI to measure how the predictability of behaviour trees can be impacted by the adaptability of utility AI.

References

- Barriaes, S.O. (2015) 'Building a Risk-Free Environment to Enhance Prototyping: Hinted-Execution Behaviour Trees', *Game AI Pro*. 2nd edition. CRC Press, pp. 69-89
- Champanard, A, Dunstan, P. (2013) 'The Behaviour Tree Starter Kit', *Game AI Pro*. 1st edition. CRC Press, pp. 73-93.
- Chen, S. (2024) 'Literature review of Application of AI in improving gaming experience: NPC behaviour and dynamic difficulty adjustment', pp. 13-14. Available at: https://lutpub.lut.fi/bitstream/handle/10024/167834/mastersthesis_Chen_Shuaopdf?sequence=1&isAllowed=y (Accessed: 08 November 2025)
- Colledanchise, M. Ögren, P. (2018) 'A Short History and Motivation of BTs' *Behaviour Trees in Robotics and AI An Introduction*. 1st edition. CRC Press, pp. 4.
- Dawe, M. Gargolinski. S, Dicken. L, Humphreys. T, Mark D. (2019) 'Behaviour Selection Algorithms an Overview', *Game AI Pro*. 1st edition. CRC Press, pp. 52-53.
- Dawe, M. Gargolinski. S, Dicken. L, Humphreys. T, Mark D. (2019) 'Behaviour Selection Algorithms an Overview', *Game AI Pro*. 1st edition. CRC Press, pp. 53-55.
- Dill, K. and Martin, L., 2011. A game AI approach to autonomous control of virtual characters. In *Interservice/Industry Training, Simulation, and Education Conference (IIITSEC)* (Vol. 2011, pp. 1-11). Available at: https://course.ccs.neu.edu/cs5150f13/readings/dill_granny.pdf (Accessed: 13 March 2026)
- Graham, D (2013) 'An Introduction to Utility Theory', *Game AI Pro*. 1st edition. CRC Press, pp. 113-127.

Millington, I. Fudge, J. (2019) 'Decision Making', *AI For games*. 3rd edition. CRC Press, pp. 293.

Millington, I. Fudge, J. (2019) 'Decision Making', *AI For games*. 3rd edition. CRC Press, pp. 334-370.

Millington, I. Fudge, J. (2019) 'Decision Making', *AI For games*. 3rd edition. CRC Press, pp. 459-466.

Mark, D (2009) 'The Concept of Utility', *Behavioural Mathematics for game AI*. Cengage Learning PTR, pp. 111-167.

Mark, D (2009) 'Marginal Utility', *Behavioural Mathematics for Game AI*. Cengage Learning PTR, pp. 167-195.

Maxis (2000 – 2017) *The Sims Video Games* [Video Games]. Electronic Arts. Available at: <https://www.ea.com/en-gb/games/the-sims> (Accessed: 02 March 2026)

Epic Games (2025) *Unreal Engine* (version 5.6) [Computer Program]. Available at: <https://dev.epicgames.com/documentation/en-us/unreal-engine/install-unreal-engine> (Accessed: 28 February 2026)

Epic Games (2025) *Behaviour Tree Overview*. Available at: <https://dev.epicgames.com/documentation/en-us/unreal-engine/behavior-tree-in-unreal-engine---overview> (Accessed: 13 February 2026)

Epic Games (2025) *Logging in Unreal Engine*. Available at: <https://dev.epicgames.com/documentation/en-us/unreal-engine/logging-in-unreal-engine> (Accessed: 27 February 2026)

Epic Games (2025) *State Tree Utility Selectors*. Available at: <https://dev.epicgames.com/documentation/en-us/unreal-engine/state-tree-selectors-overview> (Accessed: 01 March 2026)

Francis, A. G. 2000. Context-sensitive asynchronous memory. PhD diss. Georgia Institute of Technology. Available at: <http://dresan.com/research/publications/thesis.pdf> (Accessed: 03 March 2026).

Francis, A. (2017) 'Overcoming Pitfalls in Behaviour Tree Design'. *Game AI Pro*. 3rd edition. CRC Press, pp. 115-127

Isla, D. (2005) *Handling Complexity in the Halo 2 AI* [GDC Presentation] Game Developers Conference, San Francisco. Available at: <https://gdcvault.com/play/1020270/Managing-Complexity-in-the-Halo> (Accessed: 01 March 2026)

Kasurinen. J and Smolander. K (2014) 'What do game developers test in their products'. *ESEM '14: Proceedings of the 8th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, pp. 1-10. Available at: <https://dl.acm.org/doi/pdf/10.1145/2652524.2652525> (Accessed: 05 March, 2026)

Lim, CU. Baumgarten, R. Colton, S. (2010). Evolving Behaviour Trees for the Commercial Game DEFCON. *Applications of Evolutionary Computation*, 6024, pp. 100-110.

Available at https://link.springer.com/chapter/10.1007/978-3-642-12239-2_11
(Accessed: 01 March 2026)

Lim, M.I. Dias, J. Aylett, R. Paiva, A. (2010) 'Creating adaptive affective autonomous NPCs' *Autonomous Agents and Multi-Agent Systems*. 24, pp. 287-311. Available at: <https://link.springer.com/article/10.1007/s10458-010-9161-2> (Accessed: 23 March 2026)

Russel, S and Norvig, P. (2002) 'Making Simple Decisions' *Artificial Intelligence A Modern Approach*. 2nd edition. Prentice Hall, pp. 584-613.

Shao, Z.K (2025) *C++ Debugging tip: Data breakpoints*. Available at: <https://dev.epicgames.com/community/learning/tutorials/5V1K/unreal-engine-c-debugging-tip-data-breakpoints> (Accessed: 27 February 2026)

Simonov, A. Lebin, A. Shcherbak, B. Zagarskikh, A. Karasov, A, (2018) 'Multi-agent crowd simulation on large areas with utility-based behaviour models: Sochi Olympic Park Station use case', *Procedia Computer Science*, 136, pp. 453-462. Available at: <https://www.sciencedirect.com/science/article/pii/S1877050918315710> (Accessed: 02 March 2026)

Spronck, P. Ponsen, M. Sprinkhuizen-Kuyper, I. Postma, E. Adaptive game AI with dynamic scripting. *Mach Learn* **63**, 217–248 (2006). Available at: <https://doi.org/10.1007/s10994-006-6205-6> (Accessed: 08 November 2025)

Stainless Steel Studios (2001) *Empire Earth* [Video Game]. Sierra Entertainment. (Accessed 13 March 2026)

Uludağlı, M.C. Oğuz, K. (2023) 'Non-Player character decision-making in computer games', *Artificial Intelligence Review*, 56, pp. 14159-14191. Available at: <https://doi.org/10.1007/s10462-023-10491-7> (Accessed: 02 March 2026)

Unity Technologies (2025) *Unity Engine* (version 6.2) [Computer Program]. Available at: <https://unity.com/> (Accessed: 28 November, 2025)

Warpefelt, H (2016) 'The Non-Player Character: Exploring the believability of NPC presentation and behaviour', *Department of Computer and Systems Sciences, Stockholm University*, pp 118. Available at: <https://www.diva-portal.org/smash/record.jsf?pid=diva2%3A912617&dswid=7630> (Accessed: 28 February 2026)

